CYBRO × PESSIMISTIC

SECURITY ANALYSIS

by Pessimistic

This report is public

February 12, 2026

Abstract 3
Disclaimer 3
Summary 3
General recommendations 4
Project overview 5
Project description 5
Codebase update #1 5
Codebase update #2 6
Codebase update #3 6
Audit process 7
Manual analysis 9
Critical issues 9
Medium severity issues 10
M01. No documentation (addressed) 10
M02. Discrepancy with NatSpec comments (fixed) 10
M03. The owner's role (commented) 11
M04. Overflow (fixed) 12
M05. Tick bounds overflow (acknowledged) 13
M06. Lack of slippage protection (acknowledged) 13
Low severity issues 14
L01. Duplicate fee logic across contracts (fixed) 14
L02. Gas consumption (fixed) 14
L03. Missing events (fixed) 14
L04. Typo in the function name (fixed) 14
L05. Unused import (fixed) 15
L06. Using Ownable2Step (acknowledged) 15
L07. Zero price (acknowledged) 15
L08. Price fetch blocked due to observations buffer exhaustion (acknowledged) 15
L09. Insufficient replay protection due to unused nonce (acknowledged) 16
L10. Manipulable spot price (acknowledged) 16
L11. Faulty price rounding may disrupt fee claims (acknowledged) 16
Notes 17
N01. Direct Uniswap calls bypass LPManager logic (acknowledged) 17
N02. Typehash formatting mismatch (acknowledged) 17
N03. Inconsistency with native asset address convention (acknowledged) 17

N04. Interface and implementation mismatch (acknowledged) 18

ABSTRACT

This report evaluates the security of the smart contracts in the CYBRO Liquidity manager project. The objective was to identify and document security vulnerabilities within the platform's smart contracts.

DISCLAIMER

This audit makes no warranties or guarantees as to the security of the code. A single audit is insufficient to ensure complete security. We strongly recommend conducting multiple independent audits and implementing a public bug bounty program to enhance smart contract security. Additionally, this security audit should not be construed as investment advice.

SUMMARY

This report assesses the security of the CYBRO Liquidity manager smart contracts. The audit process is detailed in the following section.

The audit identified several medium severity issues: No documentation, Discrepancy with NatSpec comments, The owner's role, and Overflow. Also, several low severity issues were found.

Overall code quality is good.

After the initial audit, the codebase was updated. The No documentation issue of medium severity was addressed, and the Discrepancy with NatSpec comments issue of medium severity was fixed. The owner's role and Overflow issues of medium severity were commented on by the developers. All low severity issues and notes were either fixed or commented on. The test passed.

After that, the developers provided us with an updated version of the code. In this update, two new medium severity issues Tick bounds overflow and Lack of slippage protection were discovered. The existing The owner's role and Overflow issues were updated with new entries. Also, the audit identified several low severity issues: Price fetch blocked due to observations buffer exhaustion, Insufficient replay protection due to unused nonce, Manipulable spot price, and Faulty price rounding may disrupt fee claims. And besides, a few notes were added to the report.

After the second codebase update, the developers provided us with a new version of the code. The Overflow issue of medium severity was fixed. All low severity issues and notes were acknowledged by the developers. No new issues were discovered.

GENERAL RECOMMENDATIONS

We recommend resolving the remaining issues.

PROJECT OVERVIEW

Project description

For this audit, we reviewed the CYBRO LPManager project from a private GitHub repository, commit c7df4976c0e53a22a15179ebece4cead4511f63e.

The scope of the audit included:

- **LPManager.sol**;
- **ProtocolFeeCollector.sol**;
- **interfaces/ILPManager.sol**;
- **interfaces/IProtocolFeeCollector.sol**.

The list of networks for deployment:

- Arbitrum;
- Base;
- Unichain.

The project has only 1 test, that passes successfully. The code coverage is 87.93%.

The total lines of code (LOC) of audited sources are 680.

Codebase update #1

After the initial audit, the codebase was updated. For the recheck, we were provided with commit b13d407d9271a8936e51e27e9c45a1dd7c43b5b2.

The 1/1 test passed, the code coverage was 86.74%.

All issues were fixed or commented on by the developers.

Codebase update #2

After the previous codebase update, the developers provided us with a new version of the code, commit 9698e87a65fa0cee05249dbcfb648f3b03e5702a.

In this code update, the **LPManager** contract was split into two **BaseLPManagerV3** and **BaseManagerV3** contracts. The scope was also extended with new contracts.

The new scope of the audit included:

- **UniswapV3BaseLPManager.sol**;
- **UniswapV3AutoManager.sol**;
- **UniswapV3LPManager.sol**;
- **PancakeV3BaseLPManager.sol**;
- **PancakeV3AutoManager.sol**;
- **PancakeV3LPManager.sol**;
- **BaseManagerV3.sol**;
- **BaseLPManagerV3.sol**;
- **BaseAutoManagerV3.sol**;
- **ProtocolFeeCollector.sol**;
- **interfaces/IProtocolFeeCollector.sol**.

All 13 tests passed successfully. The code coverage increased to 88%.

The total lines of code (LOC) of audited sources are 1196.

Codebase update #3

After the second codebase update, the developers provided us with a new version of the code, commit e5980b5594f9a8b2699f834f99b93f70e80a8fee.

All 13 tests passed successfully, the code coverage was 84.41%.

In this code update, the developers fixed one medium severity issue and commented on all other issues.

AUDIT PROCESS

The audit was conducted from September 25 to September 30, 2025.

We began by reviewing the provided materials. Then we performed a comprehensive manual review of all contracts within the audit scope, verifying the logic and security properties, including:

- Standard Solidity issues;
- Integration with Uniswap V3;
- Fee calculation and charging logic;
- Calculation of the ratio between token0 and token1 for depositing to Uniswap V3.

We analyzed the project with the following automated tools:

- AI scanner AuditAgent;
- AI scanner Savant Chat;
- Static analyzer Slither;
- Our plugin Slitherin with an extended set of rules;
- Semgrep rules for smart contracts.

We executed the test suite and calculated code coverage.

All identified issues — whether detected manually or through automated tools — were compiled into the private report.

We performed the recheck on October 6, 2025. We checked whether the previously reported issues were fixed, re-ran the tests, and recalculated the code coverage.

After that, we updated the report.

After the recheck, the developers made updates to the codebase. Some new issues were introduced with this update.

The audit of the new codebase was conducted from February 2 to February 6, 2026.

First, we verified the correctness of splitting the **LPManager** contract into new **BaseLPManagerV3** and **BaseManagerV3** contracts, as well as potential changes to the **ProtocolFeeCollector** contract.

Next, we conducted a comprehensive manual review of all new contracts within the audit scope, verifying the logic and security properties, including:

- Standard Solidity issues;
- Integration with Uniswap V3 and PancakeSwap V3;
- Security of user-signed off-chain messages;
- Robustness of asset prices requests;
- Impact on financial outcomes for users employing deferred position modifications.

For the new scope, we also analyzed the code using automated tools:

- AI scanner AuditAgent;
- AI scanner Savant Chat;
- Static analyzer Slither;
- Our plugin Slitherin with an extended set of rules;
- Semgrep rules for smart contracts.

We ran tests and calculated code coverage. The number of tests and code coverage increased.

After that, we updated the report.

We performed the second recheck on February 12, 2026. We checked whether the previously reported issues were fixed, re-ran the tests, and recalculated the code coverage.

Also, we did not discover any new issues in this version of the code.

Finally, we updated the report.

MANUAL ANALYSIS

All contracts were manually inspected to validate their logic and security. Additionally, automated analysis results were manually verified. Confirmed issues are documented below.

Critical issues

Critical issues pose severe risks to project security, potentially leading to fund loss or other catastrophic failures. Contracts should not be deployed until such issues are resolved.

No critical issues were identified during the audit.

Medium severity issues

Medium severity issues may impact the project's current functionality. This category includes bugs, potential revenue loss, operational inefficiencies, and risks related to improper system management. We strongly recommend addressing these issues.

M01. No documentation (addressed)

The project has no documentation. Documentation is a critical part that helps to improve security and reduce risks. It should explicitly describe the purpose and behavior of the contracts, their interactions, and key design choices. Documentation is also essential for any further integrations.

As an example, due to the lack of documentation, it was unclear how certain fees are intended to be applied. Specifically, the `LPManager.moveRange` function charges `liquidityProtocolFee` on the entire position (initial principal + claimed fees). In contrast, the `compoundFees` function charges the fee only on accrued rewards.

> The developers added more detailed NatSpec comments.

M02. Discrepancy with NatSpec comments (fixed)

The `LPManager.createPosition` function charges `liquidityProtocolFee`. However, according to the NatSpec comment at line 264, it should charge the `depositProtocolFee` instead.

> The issue has been fixed and is not present in the latest version of the code.

M03. The owner's role (commented)

In the current implementation, the system depends heavily on the owner role. Thus, there are scenarios that can lead to undesirable consequences for the project and its users, e.g., if owner's private keys become compromised.

- In the **ProtocolFeeCollector** contract, the `owner` can set different types of fees through the `setFees`, `setLiquitityProtocolFee`, `setFeesProtocolFee`, `setDepositProtocolFee` functions. However, these functions do not enforce an upper bound on the fee values. As a result, the `owner` could configure fee values greater than 100%, which would effectively block the functionality of the **LPManager** contract.

  In addition, setting fees close to 100% would allow the owner to capture nearly all user assets. In the worst case, the owner could front-run a large deposit by setting the high fees and draining user funds.

  > `Comment from the developers:`
  >
  > The problem is known and will not be fixed, as the contract owner is a multisig wallet. Deployed contracts:
  >
  > - Unichain 0xfB84664E669f8aaE284C626dC7b31c4e45101348;
  > - Arbitrum 0xfD3BE1d641aDAa64C005dC2D6A2Ff5bEA96AD744;
  > - Base 0x567103a40C408B2B8f766016C57A092A180397a1.

- In the **BaseAutoManager** contract, the `DEFAULT_ADMIN_ROLE` can set the `oracle` address to a malicious contract that provides incorrect asset prices, which can trigger off-script calls of the `autoRebalance` and `autoClaimFees` functions.

  > `Comment from the developers:`
  >
  > Default admin role will be set to multisig wallet.

- In the **BaseAutoManager** contract, the `AUTO_MANAGER_ROLE` can front-run a user's `invalidateDigest` transaction to execute the `autoRebalance`, `autoClaimFees`, and `autoClose` functions despite the user's cancellation intent.

  > `Comment from the developers:`
  >
  > Acknowledged, we won't fix.

We recommend designing contracts in a trustless manner or implementing proper key management, e.g., setting up a multisig.

M04. Overflow (fixed)

In the contracts, several functions contain expressions that square `UniswapV3.sqrtPriceX96`. Since the upper maximum value of `sqrtPriceX96` is `1461446703485210103287273052203988822378723970341`, there exists a range of values where squaring will overflow the `uint256` and `uint160` types, causing transaction reverts. Entries are described below:

- In the `LPManager._toOptimalRatio` function, the expression: `uint256(prices.current) * uint256(prices.current)` may overflow and cause a revert. Since `prices.current` has `sqrtPriceX96` type. This could make it impossible to open new positions or increase liquidity in existing ones for certain price ranges, effectively limiting protocol functionality.

  > The corresponding code was relocated to the new `BaseLPManagerV3._oneToZero` and `BaseLPManagerV3._zeroToOne` functions in the Codebase update #2. The issue was fixed in the Codebase update #3 and is not present in the latest version of the code.

- In the `BaseAutoManagerV3._checkPriceManipulation` function, the expressions: `getCurrentSqrtPriceX96(poolInfo.pool) ** 2` and `trustedSqrtPrice ** 2` may overflow and cause a revert.

  This can make it impossible to rebalance a user's position via the `autoRebalance` function on a trigger. Which not only limits the protocol's functionality but can also cause financial losses for the user if the current price moves out of the user's active range (due to impermanent loss) while the user is relying on automatic rebalancing.

- In the `BaseAutoManagerV3._getPricesToUsd` function, the expression: `twap * twap` may overflow and cause a revert. This can make it impossible to obtain a valid return value from the `needsClaimFees` trigger check, as well as impossible to execute the `autoClaimFees` function.

> The issues have been fixed and are not present in the latest version of the code.

M05. Tick bounds overflow (acknowledged)

In the `BaseAutoManagerV3.autoRebalance` function, a new price range is computed based on the initial `tickLower` and `tickUpper`. The new price range equals the width between the initial ticks, and the new ticks are selected relative to the updated `currentTick` value. When rebalancing an extremely wide range, the new `tickLower` or `tickUpper` may exceed the valid tick range for UniswapV3 `[TickMath.MIN_TICK:TickMath.MAX_TICK]`, causing the function to revert.

Additionally, when rebalancing an extremely narrow range where `widthTicks == tickSpacing`, and `currentTick` is in the range `(-tickSpacing / 2 : tickSpacing / 2)`, both values converge to the same tick, resulting in `newLower >= newUpper`, which also causes the function to revert.

> `Comment from the developers:`
> Acknowledged, we won't fix.

M06. Lack of slippage protection (acknowledged)

The **BaseAutoManagerV3** contract's functions invoked by `AUTO_MANAGER_ROLE`, lack slippage protection checks during position range changes, position closures, and fee swaps:

- The `autoRebalance` function, which calls `_moveRange`, passes a hardcoded 0 value for `minLiquidity` for a new position creating.
- The `autoClose` function, which calls `_withdrawAndChargeFee → _withdraw`, uses a hardcoded 0 values for `amount0Min` and `amount1Min` params for liquidity withdrawal.
- The `autoClaimFees` function, which calls `_claimFees → _chargeFeeSwapTransfer → _swap → _swapWithPriceLimit`, passes a hardcoded 0 value for `sqrtPriceLimitX96`, limiting swap slippage only to `TickMath.MIN_TICK` or `TickMath.MAX_TICK` boundaries.

This opens up the possibility of sandwich attacks that can reduce the principal or collected fees.

> `Comment from the developers:`
> Acknowledged, we won't fix.

Low severity issues

Low severity issues do not immediately affect project operations but may introduce risks in future iterations. We recommend resolving these issues or providing justification for the chosen implementation.

L01. Duplicate fee logic across contracts (fixed)

Fee calculation (in the `calculateLiquidityProtocolFee`, `calculateFeesProtocolFee`, `calculateDepositProtocolFee` functions of the **ProtocolFeeCollector** contract) and collection logic (in the `_collectFeesProtocolFee`, `_collectLiquidityProtocolFee`, `_collectDepositProtocolFee` functions of the **LPManager** contract) is duplicated across multiple functions for different fee types (deposit, fees, liquidity). This increases code size and reduces readability. Consider creating a single function in each contract (one private in the **LPManager** contract, one external in the **ProtocolFeeCollector** contract) and passing the fee type as a parameter.

> The issue has been fixed and is not present in the latest version of the code.

L02. Gas consumption (fixed)

The `ProtocolFeeCollector.withdrawProtocolFees` function accepts a single token address per call. If the protocol collects fees in multiple tokens, the owner must call the function separately for each token. Consider adding functionality to withdraw fees for multiple tokens in a single transaction to reduce gas consumption.

> The issue has been fixed and is not present in the latest version of the code.

L03. Missing events (fixed)

In the **ProtocolFeeCollector** contract, the constructor does not emit events when setting `liquidityProtocolFee`, `feesProtocolFee`, and `depositProtocolFee` variables. This would allow easier tracking and auditing of the initial fee setup.

> The issue has been fixed and is not present in the latest version of the code.

L04. Typo in the function name (fixed)

The function name `ProtocolFeeCollector.setLiquitityProtocolFee` contains a typo. The correct spelling should be `setLiquidityProtocolFee`. The same typo exists in the **IProtocolFeeCollector** interface.

> The issue has been fixed and is not present in the latest version of the code.

L05. Unused import (fixed)

The **LPManager** contract has unused import of the **LiquidityAmounts** contract at line 12.

> The issue has been fixed and is not present in the latest version of the code.

L06. Using Ownable2Step (acknowledged)

The **ProtocolFeeCollector** contract inherits from the standard **Ownable** contract. Ownership can be transferred in a single step using `transferOwnership` function. However, single-step transfers carry a small risk of accidental or malicious ownership change if transferOwnership is called incorrectly. To mitigate this risk, consider using Ownable2Step for a safer two-step ownership transfer process.

> `Comment from the developers:`
>
> Acknowledged, we won't fix.

L07. Zero price (acknowledged)

In the `LPManager._getPriceFromPool` function, if the rate of `token1` relative to `token0` is less than 1, the numerator at line 894 may be smaller than the denominator, causing the calculated `price` to be 0. Consider multiplying the `ratio` at line 893 by the decimals of `token1` to preserve precision and avoid a zero price.

> `Comment from the developers:`
>
> Acknowledged, we won't fix.

L08. Price fetch blocked due to observations buffer exhaustion (acknowledged)

In case the `BaseAutoManager._getPricesFromOracles` function returns incorrect asset prices, the `_getPricesToUsd` and `_getTrustedSqrtPrice` functions rely on obtaining the TWAP via the `_getTwap` function directly from the UniswapV3 pool using the `observe` function with a fixed 30-minutes window. The `_getTwap` function does not handle cases where the `UniswapV3Pool.observe` function may revert due to an insufficient observations buffer, which will block execution of the `autoRebalance` and `needsClaimFees` functions.

> `Comment from the developers:`
>
> Acknowledged, we won't fix.

L09. Insufficient replay protection due to unused nonce (acknowledged)

The **BaseAutoManagerV3** contract implements EIP-712 signed requests (`AutoCloseRequest`, `AutoRebalanceRequest`) that include a `nonce` field intended to provide replay protection. The signature validation logic in `_validateSignatureFromOwner` function checks if the `digest` was explicitly marked invalid via the `invalidateDigest` function call only. Importantly, the state-changing `autoClose` and `autoRebalance` functions validate the signature but do not mark the `digest`, `signature`, or `nonce` as used in the contract's storage. Although UniswapV3 positions are often treated as one-time use, users can add liquidity back to an existing position ID. If a user does this after `autoClose` or `autoRebalance` of a position, the previous signed request remains valid. An `AUTO_MANAGER_ROLE` can reuse the old `signature` to close or rebalance the position without the user's intent.

> `Comment from the developers:`
> Acknowledged, we won't fix.

L10. Manipulable spot price (acknowledged)

The `BaseAutoManagerV3.autoClose` function relies on the instantaneous spot price by `UniswapV3Pool.slot0` to validate stop-loss triggers via the `needsClose` function, lacking any oracle price or TWAP verification. This allows attackers to use flash loans to manipulate the pool price, triggering user position closures at unfavourable rates (Stop-Loss Hunting), causing principal loss. Unlike `autoRebalance`, which includes a `_checkPriceManipulation` guard to compare the spot price against a trusted oracle price or TWAP, `autoClose` performs no such validation.

> `Comment from the developers:`
> Acknowledged, we won't fix.

L11. Faulty price rounding may disrupt fee claims (acknowledged)

In the `BaseAutoManagerV3._getPricesToUsd` function, intermediate price value `price0` in the nested `mulDiv` operation can round down to 0 in cases of small `decimals0` value (1e6) and `twap` in the approximate range `[-887272:-140000]`, or with standard `decimals0` (1e18) and `twap` in the range `[-887272:-420000]`. Similarly, intermediate `price1` value in the nested `mulDiv` can round down to 0 with small `decimals1` (1e6) and `twap` in the approximate range `[140000:887272]`, or standard `decimals1` (1e18) and `twap` in the range `[420000:887272]`. This can distort the return value of the `needsClaimFees` function, cause incorrect triggering of `autoClaimFees`, and lead to `autoClaimFees` reverting even when the execution matches the trigger conditions.

> `Comment from the developers:`
> Acknowledged, we won't fix.

Notes

N01. Direct Uniswap calls bypass LPManager logic (acknowledged)

Users can bypass the **LPManager** contract by interacting directly with Uniswap V3 position manager. In such cases, the internal logic of the **LPManager** contract may become inconsistent with the actual Uniswap position state. For example, if a user calls the `decreaseLiquidity` function directly on Uniswap:

- The withdrawn amount (principal) is not returned immediately, but recorded in Uniswap's internal `tokensOwed` mapping;
- These funds can then be collected via the `collect` function;
- When the user later calls the `compoundFees`, `moveRange`, or `claimFees` function through the **LPManager** contract, the functions may incorrectly treat this amount as accrued fees.

> `Comment from the developers:`
> Acknowledged, we won't fix.

N02. Typehash formatting mismatch (acknowledged)

In the **BaseAutoManagerV3** contract, the variables `AUTO_CLAIM_REQUEST_TYPEHASH`, `AUTO_REBALANCE_REQUEST_TYPEHASH`, and `AUTO_CLOSE_REQUEST_TYPEHASH` contain an extra space after commas `", uint256 nonce)"`, which does not conform to the standard for constructing the typehash of structured data. Standard EIP-712 implementations construct the type string canonically without such spaces after commas, resulting in a different typehash and therefore signatures that fail verification. Users relying on common wallet `eth_signTypedData_v4` flows and EIP-712 libraries may be unable to validate or generate signatures that the contract accepts.

> `Comment from the developers:`
> Acknowledged, we won't fix.

N03. Inconsistency with native asset address convention (acknowledged)

According to the ETH Address convention, the address placeholder for native assets should be `0xEeeeeEeeeEeEeeEeEeEeeEEEeeeeEeeeeeeeeEEeE` in the **ProtocolFeeCollector** contract to improve compatibility and follow common practices.

> `Comment from the developers:`
> Acknowledged, we won't fix.

N04. Interface and implementation mismatch (acknowledged)

The **ProtocolFeeCollector** contract does not implement the `withdrawETH` function declared in the **IProtocolFeeCollector** interface. Additionally, the `withdrawProtocolFees(address,recipient)` function has a different signature in the **ProtocolFeeCollector** contract.

> `Comment from the developers:`
>
> Acknowledged, we won't fix.

This analysis was performed by Pessimistic:

Daria Korepanova, Senior Security Engineer
Evgeny Bokarev, Security Engineer
Irina Vikhareva, Project Manager
Alexander Seleznev, CEO

February 12, 2026